

The roster is fixed — and what it was quietly costing you

For Michael. Written at the end of books-50. Plain English, no jargon, and every number in here was checked by a machine rather than typed by me.

The short version

You told me there are four of you on the S corporation: you and your wife at 85%, your grandfather at 5%, your mother at 5%, and your step-father at 5%.

The system believed there were **three**. It had 85% for you, **10% for "Mother"**, and 5% for your grandfather. Not four people — three, and one of them was recorded as a relationship rather than a person.

That is now corrected, everywhere, and it is locked down so it cannot drift back.

What it was worth: \$2,340. That is not a rounding difference. It is explained below.

Why nobody noticed for a year

This is the part I most want you to understand, because it is the useful lesson and it is going to come up again.

There was already a safety check watching the ownership table. Its job was to refuse any roster that does not add up to 100%. It worked perfectly. It ran every single time. It never once complained.

Look at why:

	Michael	Second holder	Third holder	Fourth holder	Total
What the system had	85%	10%	5%	—	100%
What you actually filed	85%	5%	5%	5%	100%

Both add up to exactly 100%. The check was asking "*do these numbers total 100?*" — and the answer was yes, on the wrong roster, for forty-nine rounds of work.

The lesson: a total that balances is not the same as a total that is right. The check was policing the arithmetic. Nothing was policing who the people were, or how many of them there were. A perfect total can sit on top of the wrong facts and look like health.

This is the same shape as a bank statement that reconciles to the penny while two transactions are posted to the wrong accounts. The tie-out is genuinely satisfying and genuinely tells you nothing about whether the entries are right.

What the wrong count was worth: \$2,340

There is a penalty for filing an 1120-S late. It is not a percentage of tax owed — which is what makes it dangerous for you, because an S corporation often shows no tax at all, and it is very natural to assume no tax means no penalty. It does not work that way. The penalty is a **flat amount per shareholder, per month**, for up to twelve months.

Two things follow from that, and both are worth knowing:

1. **The number of shareholders multiplies the penalty directly.** So the count in that table was not a piece of bookkeeping trivia. It was a multiplier on a real dollar figure.
2. **The percentages are completely irrelevant to it.** A 5% holder costs exactly the same as an 85% holder. Your grandfather's 5% carries the same penalty weight as your 85%.

So here is the arithmetic that was wrong:

Roster	Full-year exposure
Three shareholders (what the system believed)	\$7,020

Four shareholders (what you actually filed)	\$9,360
Understated by	\$2,340

The system had been telling you the smaller number. And it was written into the guidance as a *typed-in* figure — a sentence somebody had composed once, when three was believed to be correct.

One more thing about that \$9,360: it is the floor, not the ceiling. The underlying per-shareholder amount gets adjusted for inflation every year, and has been since 2014. So the real figure today is higher than \$9,360. I have not quoted you the current one, because I would have to look up the exact inflation-adjusted amount for the specific year to state it, and I am not going to guess a number and put it in front of you.

What I actually did about it

1. Corrected the data, without rewriting history

The original migration — the file that first created the ownership table — is left exactly as it was applied to your database. I did not go back and edit it to pretend it had always been right.

Instead there is a new step, applied on top, that corrects the roster forward. Your database moves from the wrong state to the right state by a step you can see and re-run. The old file keeps a note on it explaining that it was wrong and pointing at the correction.

Why that way: your database has already had the old file applied to it. Editing history in the repository would make the code disagree with reality, and the next person reading it — including future me — would have no way to know a correction had ever happened.

2. Made the check ask a better question

The new correction does **not** verify its work by adding up to 100%, because that is precisely the check that failed to notice anything was wrong.

It compares the roster in the database against the four people on your filed K-1s, **name by name**, in both directions at once. It refuses if there is anyone in the database who is not on the filed list, *and* it refuses if there is anyone on the filed list who is not in the database. Both directions, because checking only one of them lets an extra person hide.

3. Made the dollar figure calculate itself

That \$9,360 is no longer a sentence anybody typed. The system now works it out from the number of people on the roster, every time it shows it to you.

This matters more than it sounds. A number typed into a sentence is correct on the day it is typed and wrong forever afterwards, and it never once goes red to tell you. That is exactly how the old figure survived — nobody mistyped anything. The sentence was simply written when three was believed to be true, and it stayed there.

If a shareholder is ever added or removed, that figure now moves on its own.

4. Built a tripwire so it cannot come back

There is now a check that scans **every file in the whole project** and fails if any of them states the old roster or the old dollar figure as though it were true. Not just the accounting code — documents, tests, comments, all of it.

It found nine places on its first run that I had missed. It also, correctly, flagged my own new writing, because the sentence "it used to say 10%" looks identical to the sentence "it is 10%" as far as a machine is concerned. So there is a way to mark a line as *history* — but the mark has to sit on the exact line it excuses, so anybody reading that line sees it.

How hard I tried to break it

I want to be specific here, because "I tested it" is worth very little on its own.

I built the bug on purpose before fixing it. The verification starts by applying the *original* migration to a fresh database, so the wrong three-person roster really exists — then asserts it really is three rows, really has someone at 10%, really has a row

named "Mother", and really does total exactly 100%.

That last one is the important assertion. It is the proof that the old safety check could never have caught this.

Without this step the whole exercise would have been theatre: inserting a correct roster into an empty table and then checking that it is correct proves nothing at all.

Then I attacked the fix:

- Applied it **three times in a row**. You apply these by hand, and a hand can slip. Twice is not enough, because a bug that alternates passes a second run.
- **Put the wrong roster back** and ran the fix again. It corrected it again.
- **Deleted every shareholder** and ran the fix. It rebuilt all four from nothing.
- **Deleted the company record itself** and ran the fix. It refused, with a clear message, and — this is the part that matters — it wrote *nothing at all* while refusing. A safety check that half-does the job before complaining is worse than one that does nothing.

I also applied **all 205 migration steps in order, from empty, to a real database**, and then checked the roster at the end. It reads 85/5/5/5, four people, totalling 100%.

And I found a second hole in that same old safety check while I was in there. If you delete *every* shareholder, it raises no complaint whatsoever — an empty table has no total to disagree with. So "all four shareholders vanish" is currently a permitted operation. I have written that down, with a test that records it, and I have **not** fixed it in this round, because changing that check touches the database structure and belongs in its own piece of work. It is on the list, not in a comment.

Two mistakes I made, and what they teach

I would rather tell you these than have you find them.

One: my own verification script was broken in the way it was documented to be run. Its instructions said to run it one way, and run exactly that way, it died on its very first command. Worse, it died in a place where the failure could look like a success to someone skimming the output. Eleven other scripts of mine in the same folder had solved that problem long ago — I had written a twelfth that did not.

The lesson: the tool that checks your work is also code, and it is the code least likely to be checked, because when it passes nobody looks at it and when it fails everyone blames the thing being tested.

Two: one of my tests was defending the bug. There was a test that *required* the guidance to contain "\$7,020". It had been written to prove the system named a specific dollar amount instead of vaguely gesturing at "a large penalty" — a good instinct. But it had quietly turned into a test that would fail the day somebody corrected the figure. It was voting for the wrong number.

That one is worth sitting with. A test is not automatically on your side.

The bit I found genuinely reassuring

Correcting this one fact turned **18 tests red** across four different files.

That sounds like damage. It is the opposite — it is a map. Those tests were recounting figures from the code and telling me exactly which sentences, in which documents, had gone stale. The lesson count on your learning screen, the totals in the roadmap, two documents written for you, and a PDF that is a rendering of one of those documents — every one of them named itself as out of date.

A system where correcting one fact turns 18 tests red is a system where nobody can correct one fact and leave seventeen lies behind.

Everything is green now: **449 test files, 11,046 tests, all passing**, plus the type checker, the linter, the authority-quote verifier, and the full 205-migration run — all clean, both on my branch and again after it was merged.

Where the four of you now sit in the system

Name	Relationship	Ownership	Recorded as receiving distributions?
Michael B Lyman	owner	85%	yes
Nicholas C Mullan	grandfather	5%	yes
Theresa L Becker	mother	5%	flagged — please confirm
James H Becker	step-father	5%	flagged — please confirm

Totals 100%. Four shareholders, which is the number that drives the penalty arithmetic.

On your wife: you said "my wife and I: 85%". The system holds that 85% as a single interest in your name, which is how it appeared on the filed K-1s. For the penalty count that is the right answer regardless, because a husband and wife are treated as one shareholder for this purpose. I have not invented a fifth row.

Three things I need from you

Nothing here is urgent. None of it blocks the next piece of work.

- 1. Do your mother and step-father actually receive distributions?** The system currently says no for both, and I have marked that as unconfirmed rather than letting it look like a fact. It does not affect the penalty count at all. It *does* matter when we get to distributions and basis, because if money is paid out in proportions that differ from ownership, that is a question worth getting right early rather than late.
- 2. The ATM company's legal name, and the land-holding company's legal name.** You said you would ask your grandfather. Still outstanding, still not guessed.
- 3. A correction to something I told you earlier.** I previously implied the 2027 rates were urgent. They are not — you told me they are not published until around November, and it is August. I was wrong to frame that as pressing, and I would rather say so plainly than let it sit.

One thing I should flag from your Schedule C

While reconciling names against your filed returns I noticed something I do not want to leave unmentioned.

There are two Schedule Cs. The first is named like the **land-holding** business but carries the industry code for **ATM / financial services**. The second is named like the **cannabis/glass** business but carries the industry code for **real estate rental** — and shows exactly \$48,000 of receipts, which looks like rent.

Those two codes appear to be **swapped**. And separately: rent is being reported on Schedule C rather than Schedule E, which is not a neutral choice — income on Schedule C is generally subject to self-employment tax, and rental income on Schedule E generally is not.

I am **not** telling you it is wrong. Your grandfather prepares these, he knows facts about the arrangement that I do not, and there may be a deliberate reason. But it is the kind of thing that is cheap to ask about now and expensive to discover later, so please raise it with him.

What is next

You said: *"Let's fix the roster first, then get back to building the books."*

The roster is fixed, verified, merged, and re-verified after merging. So the next piece of work is back on the books.

Two candidates, and I would like your steer:

- Close the second hole in that ownership check** — the one where deleting every shareholder raises no complaint. Small, self-contained, and it removes a real gap I found rather than one I imagined.
- Model the rent between your companies.** The cannabis business and the ATM business both rent from the land-holding business, and none of that is currently represented. It connects directly to the Schedule C question above.

My instinct is the ownership check first, because it is small and it closes a hole that is open right now. But the rent is the one

with more money attached to it, so tell me which you would rather have.